

Задача 1.(2.5 балла)

Представьте, что вам предложили обучать линейный классификатор на следующий функционал (назовём его *сферическим*):

$$Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} \log(1 + \exp(-\beta y_i \cos \theta(w, x_i))),$$

где $\theta(w, x_i)$ — угол между векторами w и x_i , $\beta > 0$ — скалярный гиперпараметр.

Ответьте на вопросы о свойствах данного функционала (в каждом пункте ожидается, что вы дадите обоснование своему ответу):

1. Легко заметить, что здесь функция потерь очень похожа на логистическую. Попробуйте максимально свести сферический функционал к логистическому — так, чтобы в нём тоже появилось скалярное произведение весов на признаки. Проанализируйте полученную формулу: в чём ключевое отличие сферического функционала от логистического?
2. Проведём более глубокое исследование. Известно, что при обучении градиентным спуском линейной модели на логистическую функцию потерь есть проблема. Рассмотрим линейно разделимую выборку с бинарной целевой переменной. На каком векторе весов будет достигаться минимум функционала с логистической функцией потерь? Решается ли эта проблема в сферическом функционале?
3. Как параметр β влияет на поведение сферического функционала? Как будет вести себя функционал при очень больших и при очень маленьких значениях β ?

1.

Вспомним формулу для косинуса через скалярное произведение:

$$\cos \theta(w, x_i) = \frac{\langle w, x_i \rangle}{\|w\| \cdot \|x_i\|}$$

Подставим в функционал:

$$Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} \log \left(1 + \exp \left(-\beta y_i \cdot \frac{\langle w, x_i \rangle}{\|w\| \cdot \|x_i\|} \right) \right)$$

А стандартный логистический функционал:

$$Q_{\log}(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} \log(1 + \exp(-y_i \langle w, x_i \rangle))$$

Получается, что в сферическом функционале вместо скалярного произведения $\langle w, x_i \rangle$ стоит его *нормированная версия* $\frac{\langle w, x_i \rangle}{\|w\| \cdot \|x_i\|}$, домноженная на константу β

Таким образом, сферический функционал можно записать как логистическую функцию потерь, применённую к нормализованным векторам:

$$Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} \log \left(1 + \exp \left(-y_i \cdot \beta \cdot \left\langle \frac{w}{\|w\|}, \frac{x_i}{\|x_i\|} \right\rangle \right) \right)$$

Ключевое отличие заключается в том, что в логистическом функционале отступ равен $M_i = y_i \langle w, x_i \rangle$ и зависит от нормы w , а в сферическом функционале отступ равен $M_i = \beta y_i \cos \theta(w, x_i)$ и **не зависит** от нормы вектора весов $\|w\|$ и нормы объектов $\|x_i\|$. Функционал *инвариантен* относительно масштабирования w . Роль «масштаба» отступа берёт на себя гиперпараметр β , сам отступ ограничен: $\cos \theta \in [-1, 1]$, поэтому $M_i \in [-\beta, \beta]$

2.

Рассмотрим логистический функционал

$$Q_{\log}(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} \log(1 + \exp(-y_i \langle w, x_i \rangle))$$

Для линейно разделимой выборки существует вектор $\hat{w}$ такой, что $y_i \langle \hat{w}, x_i \rangle > 0$ для всех i . Тогда, рассматривая $w = c\hat{w}$ при $c \rightarrow +\infty$, получаем:

$$y_i \langle c\hat{w}, x_i \rangle = c \cdot y_i \langle \hat{w}, x_i \rangle \rightarrow +\infty \implies Q_{\log}(c\hat{w}) \rightarrow 0$$

Однако значение $Q_{\log} = 0$ **не достигается** ни при каком конечном w (поскольку $\log(1 + e^{-z}) > 0$ при любом конечном z). Таким образом, инфимум функционала равен нулю, но минимум **не достигается**: $\|w^*\| = +\infty$.

При обучении градиентным спуском веса w растут неограниченно ($\|w\| \rightarrow \infty$), что приводит к численным проблемам и отсутствию единственного решения (направление $w/\|w\|$ стабилизируется, но сама норма не ограничена)

В сферическом функционале отступ равен $\beta \cos \theta(w, x_i)$ и не зависит от $\|w\|$

Масштабирование вектора весов $w \mapsto cw$ ($c > 0$) не меняет значение функционала, так как

$$\cos \theta(cw, x_i) = \frac{\langle cw, x_i \rangle}{\|cw\| \cdot \|x_i\|} = \frac{c \langle w, x_i \rangle}{c \|w\| \cdot \|x_i\|} = \cos \theta(w, x_i)$$

Следовательно, увеличение нормы w не уменьшает функционал. Минимум ищется фактически по направлению $w/\|w\|$ на единичной сфере, которая является компактным множеством. По теореме Вейерштрасса непрерывная функция на компакте **достигает** своего минимума.

Таким образом, **проблема решается**: в сферическом функционале минимум всегда достигается на конечном векторе весов, на конечном направлении

3.

Параметр β стоит как множитель перед $\cos \theta$:

$$Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} \log(1 + \exp(-\beta y_i \cos \theta(w, x_i)))$$

Поскольку $\cos \theta \in [-1, 1]$, то аргумент экспоненты лежит в $[-\beta, \beta]$

- $\beta \rightarrow 0$: При малых β аргумент экспоненты стремится к нулю для всех объектов:

$$\log(1 + e^{-\beta y_i \cos \theta}) \approx \log(1 + 1 - \beta y_i \cos \theta) \approx \log 2$$

Функционал становится практически константным ($Q \approx \log 2$) и перестаёт различать правильные и неправильные классификации. Градиенты становятся малыми, оптимизация затрудняется. Функционал становится «нечувствительным» к выбору w .

- $\beta \rightarrow +\infty$: Потеря для правильно классифицированного объекта ($y_i \cos \theta > 0$):

$$\log(1 + e^{-\beta \cdot (\text{полож.})}) \rightarrow \log(1 + 0) = 0$$

Потеря для неправильно классифицированного объекта ($y_i \cos \theta < 0$):

$$\log(1 + e^{\beta \cdot |\cdot|}) \approx \beta |y_i \cos \theta| \rightarrow +\infty$$

Функционал начинает очень жёстко «штрафовать» неправильно классифицированные объекты и функция потерь приближается к хинжевой с масштабом β . Решение становится менее «мягким» — классификатор стремится к полному разделению. При больших β задача становится всё более похожей на максимизацию углового margin.

Таким образом, β играет роль **температуры**: малые β делают функционал «мягким» и нечувствительным, большие β делают его «жёстким» и чувствительным к ошибкам.

Задача 2.(2.5 балла)

Ответьте на вопросы по решающим деревьям и композициям:

1. Допустим, мы решили использовать следующую функцию хаотичности (impurity) при построении решающего дерева:

$$H(R) = \max_{(x_i, y_i) \in R} y_i.$$

Чем она плоха? Какие разбиения будут предпочитаться при обучении дерева?

2. Представьте, что мы решили обучать градиентный бустинг так:

$$\frac{1}{\ell} \sum_{i=1}^{\ell} (b_N(x_i) - s_i)^2 \rightarrow \min; \quad s_i = \sum_{n=1}^{N-1} \left. \frac{\partial L(y_i, z)}{\partial z} \right|_{z=b_n(x_i)}$$

Чем плох такой подход? Попробуйте подумать, как будут меняться s_i по мере роста N . Исправьте приведённые выше формулы, чтобы получилась стандартная процедура градиентного бустинга.

3. Известно, что градиентный бустинг является в некотором смысле аналогом градиентного спуска: каждая базовая модель приближает антиградиент ошибки на обучающей выборке. А как модифицировать бустинг, чтобы он стал аналогом mini-batch градиентного спуска? Запишите формулы для сдвигов и для обучения базовой модели в этом случае. Поясните, почему это правда будет аналог.

1.

Рассмотрим задачу регрессии или классификации с непрерывными метками. Данная функция хаотичности равна максимальному значению целевой переменной в узле R

Проблемы:

Здесь нет учёта разнородности, так как хорошая impurity-функция должна отражать разброс/неоднородность в узле (например, дисперсия для регрессии, энтропия/джини для классификации). Функция $\max y_i$ зависит *только от одного экстремального объекта* и не учитывает распределение целевых переменных остальных объектов

При построении дерева мы минимизируем взвешенную сумму impurity дочерних узлов:

$$\frac{|R_L|}{|R|} H(R_L) + \frac{|R_R|}{|R|} H(R_R) \rightarrow \min$$

Подставляя $H(R) = \max y_i$:

$$\frac{|R_L|}{|R|} \max_{(x_i, y_i) \in R_L} y_i + \frac{|R_R|}{|R|} \max_{(x_i, y_i) \in R_R} y_i \rightarrow \min$$

Такая функция будет стремиться **отделить объекты с большими y_i в как можно меньшее подмножество**. Объект с наибольшим y_i будет «вытеснен» в дочерний узел с наименьшим размером. Фактически дерево будет пытаться изолировать объекты-выбросы (с наибольшими y), а не строить осмысленное разбиение

Отсутствие симметрии, ведь функция не реагирует на минимальные значения y , а значит не штрафует за наличие разнородных (по y) объектов с малыми значениями

Не уменьшается при идеальном разбиении: Если в узле значения y одинаковы, $H = y_{\max}$ будет ненулевым, в отличие от дисперсии или энтропии, которые обнуляются при однородности. Это значит, что дерево не имеет стимула достигать однородности по y в узлах

2.

Дана формулировка:

$$\frac{1}{\ell} \sum_{i=1}^{\ell} (b_N(x_i) - s_i)^2 \rightarrow \min_{b_N}; \quad s_i = \sum_{n=1}^{N-1} \left. \frac{\partial L(y_i, z)}{\partial z} \right|_{z=b_n(x_i)}$$

В правильном градиентном бустинге на N -м шаге мы хотим, чтобы очередной базовый алгоритм b_N аппроксимировал **антиградиент** функции потерь, вычисленный **в текущем приближении** $a_{N-1}(x_i) = \sum_{n=1}^{N-1} b_n(x_i)$, а не сумму градиентов в отдельных $b_n(x_i)$.

Проблема 1: Сдвиги s_i вычисляются как *сумма градиентов, взятых в разных точках* $z = b_n(x_i)$, вместо одного градиента, взятого в точке текущей композиции $z = a_{N-1}(x_i)$.

Проблема 2: Знак. Мы должны обучать базовую модель на *антиградиент* (со знаком минус).

При росте N , поскольку s_i — сумма $N - 1$ градиентов, её абсолютное значение может неограниченно расти с ростом N . Базовый алгоритм b_N (обычно дерево ограниченной глубины) не сможет аппроксимировать такие растущие значения, и обучение будет деградировать.

Правильные формулы:

Пусть $a_{N-1}(x_i) = \sum_{n=1}^{N-1} b_n(x_i)$ — текущее приближение, тогда сдвиги:

$$s_i = - \left. \frac{\partial L(y_i, z)}{\partial z} \right|_{z=a_{N-1}(x_i)} = - \left. \frac{\partial L(y_i, z)}{\partial z} \right|_{z=\sum_{n=1}^{N-1} b_n(x_i)}$$

Базовая модель обучается приближать эти антиградиенты:

$$b_N = \arg \min_b \frac{1}{\ell} \sum_{i=1}^{\ell} (b(x_i) - s_i)^2$$

Обновление композиции с learning rate η :

$$a_N(x) = a_{N-1}(x) + \eta \cdot b_N(x)$$

3.

В mini-batch градиентном спуске на каждом шаге градиент вычисляется не по всей выборке, а по случайному подмножеству $B_N \subset \{1, \dots, \ell\}$, $|B_N| = m$

Аналог в бустинге — стохастический градиентный бустинг:

На каждой итерации N

1. Выбираем случайное подмножество $B_N \subset \{1, \dots, \ell\}$, $|B_N| = m$
2. Вычисляем сдвиги **только для объектов из B_N :**

$$s_i = - \left. \frac{\partial L(y_i, z)}{\partial z} \right|_{z=a_{N-1}(x_i)}, \quad i \in B_N$$

3. Обучаем базовую модель **только по объектам из B_N :**

$$b_N = \arg \min_b \frac{1}{|B_N|} \sum_{i \in B_N} (b(x_i) - s_i)^2$$

4. Обновляем:

$$a_N(x) = a_{N-1}(x) + \eta b_N(x)$$

В обычном градиентном бустинге мы аппроксимируем антиградиент функционала $Q = \frac{1}{\ell} \sum_{i=1}^{\ell} L(y_i, a(x_i))$ по всем объектам — это аналог полного (batch) градиентного спуска в пространстве значений модели на обучающей выборке.

Если же мы берём случайное подмножество B_N , то мы аппроксимируем *стохастический антиградиент*

$$- \frac{1}{|B_N|} \sum_{i \in B_N} \left. \frac{\partial L(y_i, z)}{\partial z} \right|_{z=a_{N-1}(x_i)}$$

который является несмещённой оценкой полного антиградиента

Это в точности **аналогия с mini-batch SGD**, где вместо полного градиента используется его оценка по подвыборке. Это вносит стохастичность, которая действует как регуляризация и ускоряет обучение.

Задача 3. (2.5 балла)

Тимофей поручил студентам специализации «Машинное обучение и приложения» разработать модель для классификации состояния стойл на ферме на чистые и нуждающиеся в уборке. Стоимость работ по уборке стойла составляет 200 рублей. Если стойло ошибочно было классифицировано как не нуждающееся в уборке, ферма теряет скот и несёт убытки в 1000 рублей.

В обучающей выборке дано 8 стойл, для которых моделью были выданы следующие вероятности того, что они нуждаются в ремонте или уборке:

$b(x)$	0.1	0.1	0.4	0.4	0.4	0.8	0.8	0.95
y	0	0	0	1	0	1	0	1

Выполните задания:

1. По обучающей выборке подберите порог классификации, который соответствует наименьшим издержкам фермы. Рассчитайте Ассигасу и F-меру для обучающей выборки, исходя из выбранного вами порога.
2. Вычислите AUC-ROC для $b(x)$ на обучающей выборке. Если у нескольких объектов выход модели совпадает и равен p , то в ROC-кривой соединяйте отрезком точки, соответствующие порогам $p - \varepsilon$ и $p + \varepsilon$.
3. Допустим теперь, что мы имеем K состояний стойла. Запишите функцию потерь для этой задачи многоклассовой классификации, и предложите, как сделать так, чтобы в ней учитывались различные издержки c_{km} за выдачу класса m при правильном классе k .
4. Предложите обобщение AUC-ROC для задачи классификации с тремя классами. Объясните, как вычислять такую метрику.

Подсказка: аналитическая формула AUC-ROC выглядит так:

$$\frac{1}{\ell_- \ell_+} \sum_{\substack{i,j: \\ y_i=1, y_j=0}} ([b(x_i) > b(x_j)] + \frac{1}{2}[b(x_i) = b(x_j)])$$

где ℓ_- , ℓ_+ — количество объектов отрицательного и положительного классов. Попробуйте придумать, как в этой формуле изменить множество, по которому идёт суммирование, и как поменять нормировку, чтобы формула подходила для многоклассового случая.

1.

При пороге t мы предсказываем $\hat{y}_i = [b(x_i) \geq t]$

- Положительный класс ($\hat{y} = 1$): стойло нуждается в уборке
- FP: стойло чистое, но мы убираем $\Rightarrow$ издержки 200 руб
- FN: стойло грязное, но мы не убираем $\Rightarrow$ убытки 1000 руб
- TP и TN — без дополнительных ошибочных издержек, ведь TP правильная уборка — можно считать что стоимость уборки неизбежна, мы минимизируем *ошибочные* издержки

Ожидаемые ошибочные издержки:

$$C(t) = 200 \cdot \text{FP}(t) + 1000 \cdot \text{FN}(t)$$

Рассмотрим все осмысленные пороги:

Порог $t > 0.95$ (все предсказания 0):

TP=0, FP=0, FN=3 (пропускаем все 3 грязных), TN=5 $C = 200 \cdot 0 + 1000 \cdot 3 = 3000$

Порог $t = 0.95$ ($b(x) \geq 0.95 \Rightarrow$ предсказываем 1):

Объекты с $b(x) \geq 0.95$: объект 8 ($y = 1$): $\hat{y} = (0, 0, 0, 0, 0, 0, 0, 1)$

TP=1, FP=0, FN=2, TN=5, $C = 200 \cdot 0 + 1000 \cdot 2 = 2000$

Порог $t = 0.8$ ($b(x) \geq 0.8$):

Объекты: 6($y = 1$), 7($y = 0$), 8($y = 1$). $\hat{y} = (0, 0, 0, 0, 0, 1, 1, 1)$

TP=2, FP=1, FN=1, TN=4, $C = 200 \cdot 1 + 1000 \cdot 1 = 1200$

Порог $t = 0.4$ ($b(x) \geq 0.4$):

Объекты: 3($y = 0$), 4($y = 1$), 5($y = 0$), 6($y = 1$), 7($y = 0$), 8($y = 1$)

TP=3, FP=3, FN=0, TN=2, $C = 200 \cdot 3 + 1000 \cdot 0 = 600$

Порог $t = 0.1$ ($b(x) \geq 0.1$):

Все объекты предсказаны как 1:

TP=3, FP=5, FN=0, TN=0, $C = 200 \cdot 5 + 1000 \cdot 0 = 1000$

Вывод: минимальные издержки $C = 600$ достигаются при пороге $t = 0.4$

Accuracy и F-мера при $t = 0.4$:

Confusion matrix: TP=3, FP=3, FN=0, TN=2

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{3 + 2}{3 + 2 + 3 + 0} = \frac{5}{8} = 0.625$$

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{3}{3 + 3} = \frac{1}{2} = 0.5$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{3}{3 + 0} = 1$$

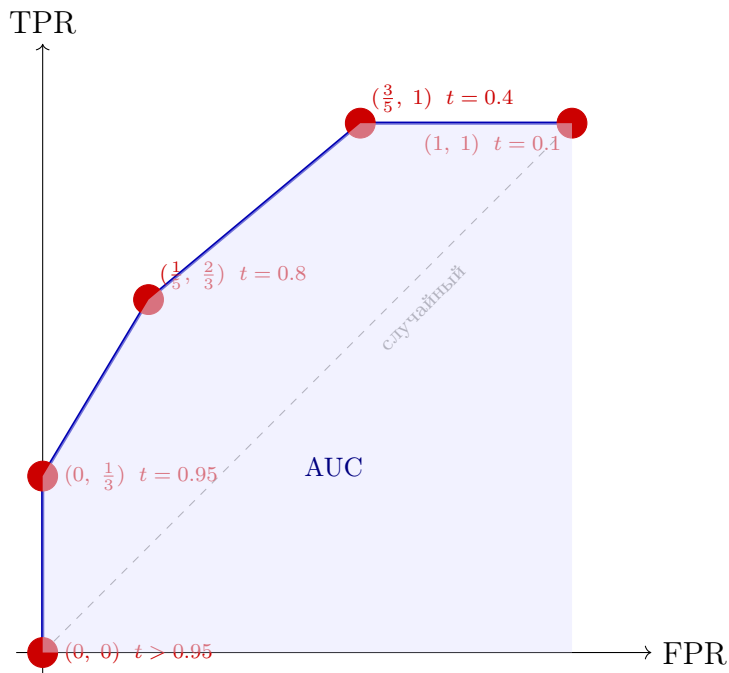
$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \cdot \frac{0.5 \cdot 1}{0.5 + 1} = \frac{1}{1.5} = \frac{2}{3} \approx 0.667$$

2.

Для построения ROC-кривой перебираем пороги от $+\infty$ до $-\infty$ и для каждого вычисляем FPR и TPR. В выборке: $\ell_+ = 3$, $\ell_- = 5$

Уникальные значения $b(x)$: 0.95, 0.8, 0.4, 0.1

По условию, при совпадающих значениях $b(x)$ мы соединяем точки для порогов $p - \varepsilon$ и $p + \varepsilon$ отрезком.



AUC методом трапеций:

$$\text{AUC-ROC} = 0 + \frac{1}{10} + \frac{1}{3} + \frac{2}{5} = \frac{3 + 10 + 12}{30} = \frac{25}{30} = \frac{5}{6} \approx 0.833$$

Проверка аналитической формулой:

$$\text{AUC} = \frac{1}{\ell_+ \ell_-} \sum_{\substack{i,j: \\ y_i=1, y_j=0}} ([b(x_i) > b(x_j)] + \frac{1}{2}[b(x_i) = b(x_j)])$$

Положительные объекты: 4($b = 0.4$), 6($b = 0.8$), 8($b = 0.95$)

Отрицательные объекты: 1($b = 0.1$), 2($b = 0.1$), 3($b = 0.4$), 5($b = 0.4$), 7($b = 0.8$)

$\ell_+ \cdot \ell_- = 3 \cdot 5 = 15$ пар. Считаем вклад каждой:

(+) \ (-)	1(0.1)	2(0.1)	3(0.4)	5(0.4)	7(0.8)	сумма
4(0.4)	1	1	1/2	1/2	0	3
6(0.8)	1	1	1	1	1/2	4.5
8(0.95)	1	1	1	1	1	5
						12.5

$$\text{AUC} = \frac{12.5}{15} = \frac{25}{30} = \frac{5}{6} \approx 0.833$$

3.

Пусть имеется K классов. Модель выдаёт вероятности $p_k(x) = P(y = k|x)$ для $k = 1, \dots, K$. Введём матрицу издержек $C = (c_{km})_{K \times K}$, где c_{km} — стоимость ошибки при предсказании класса m , когда истинный класс k .

Стандартная кросс-энтропия:

$$L(y, p) = - \sum_{k=1}^K [y = k] \log p_k(x) = - \log p_y(x)$$

Из-за издержек нужна модификация, например, есть подход — использовать **cost-sensitive** кросс-энтропию с весами, зависящими от класса и ошибки:

$$L(y, p) = - \sum_{k=1}^K [y = k] \sum_{m=1}^K c_{km} p_m(x)$$

Однако более стандартный подход — учитывать издержки на этапе принятия решения. Модель обучается предсказывать вероятности $p_k(x)$, а *решающее правило* строится так:

$$\hat{y}(x) = \arg \min_{m=1, \dots, K} \sum_{k=1}^K c_{km} p_k(x) = \arg \min_m E_{y|x} [c_{y,m}]$$

То есть мы предсказываем тот класс, который минимизирует ожидаемые издержки.

Альтернативный подход — **взвешенная кросс-энтропия:**

$$L(y, p) = -w_y \log p_y(x)$$

где $w_k = \sum_{m \neq k} c_{km}$ — суммарная стоимость ошибок для класса k

4.

Для бинарной классификации AUC-ROC — это вероятность того, что случайный положительный объект будет ранжирован выше случайного отрицательного:

$$\text{AUC} = \frac{1}{\ell_- \ell_+} \sum_{\substack{i,j: \\ y_i=1, y_j=0}} ([b(x_i) > b(x_j)] + \frac{1}{2}[b(x_i) = b(x_j)])$$

Обобщение для $K = 3$ классов

Используем подход **one-vs-one**: для каждой пары классов (k, m) , $k \neq m$, вычисляем AUC, рассматривая задачу «отличить класс k от класса m » с использованием скоринга $b_k(x) - b_m(x)$ (или $b_k(x)$, если модель выдаёт скоры для каждого класса)

$$\text{AUC}_{k,m} = \frac{1}{\ell_k \ell_m} \sum_{\substack{i,j: \\ y_i=k, y_j=m}} ([b_k(x_i) > b_k(x_j)] + \frac{1}{2}[b_k(x_i) = b_k(x_j)])$$

Итоговая метрика — усреднение по всем парам классов:

$$\text{AUC}_{\text{multi}} = \frac{2}{K(K-1)} \sum_{k < m} \text{AUC}_{k,m} = \frac{1}{3} (\text{AUC}_{1,2} + \text{AUC}_{1,3} + \text{AUC}_{2,3})$$

Как изменить аналитическую формулу:

- Множество суммирования: вместо пар (i, j) с $y_i = 1, y_j = 0$ берём все пары (i, j) с $y_i \neq y_j$ (или фиксируем пару классов и суммируем по объектам этих двух классов).
- Нормировка: вместо $\ell_- \ell_+$ — для каждой пары классов (k, m) нормируем на $\ell_k \cdot \ell_m$, затем усредняем по $\binom{K}{2}$ парам.

Альтернатива — подход **one-vs-all**: для каждого класса k вычислять AUC как «класс k vs остальные» и усреднять.

Задача 4. (2.5 балла)

Пусть логарифм обменного курса в день t изменяется на валютном рынке согласно закону

$$r_t = 0.8 r_{t-1} + 0.6 \varepsilon_t \quad r_0 = \varepsilon_0$$

Все случайные величины ε_t независимы и имеют распределение $\mathcal{N}(0, 1)$

Биржевой аналитик Одэхингум знает этот закон и для построения модели предсказания курса на следующий день использует метод обучения

$$\mu(X)(r_{t-1}) = 0.8 r_{t-1} + 0.6 \gamma,$$

где число γ семплируется из стандартного нормального распределения независимо при каждом обучении модели. Найдите шум, смещение и разброс метода обучения μ при предсказании r_2 .

Для вашего удобства напомним формулу разложения ошибки:

$$L(\mu) = \underbrace{E_{x,y}[(y - E[y | x])^2]}_{\text{шум}} + \underbrace{E_x[(E_X[\mu(X)] - E[y | x])^2]}_{\text{смещение}} + \underbrace{E_x[E_X[(\mu(X) - E_X[\mu(X)])^2]]}_{\text{разброс}}$$

Раскроем

$$r_0 = \varepsilon_0, \tag{1}$$

$$r_1 = 0.8 \varepsilon_0 + 0.6 \varepsilon_1, \tag{2}$$

$$r_2 = 0.8 r_1 + 0.6 \varepsilon_2 = 0.8(0.8 \varepsilon_0 + 0.6 \varepsilon_1) + 0.6 \varepsilon_2 = 0.64 \varepsilon_0 + 0.48 \varepsilon_1 + 0.6 \varepsilon_2. \tag{3}$$

Здесь в контексте Bias-Variance разложения: $x = r_1$ — входной признак, $y = r_2$ — целевая переменная, X — обучающая выборка, здесь определяется значением γ

Условное математическое ожидание $E[y|x] = E[r_2|r_1]$

$$r_2 = 0.8 r_1 + 0.6 \varepsilon_2$$

Поскольку ε_2 независим от r_1 :

$$E[r_2|r_1] = 0.8 r_1 + 0.6 \cdot E[\varepsilon_2] = 0.8 r_1$$

Шум

$$\text{Noise} = E_{x,y}[(y - E[y|x])^2] = E[(r_2 - 0.8 r_1)^2] = E[(0.6 \varepsilon_2)^2] = 0.36 E[\varepsilon_2^2] = 0.36$$

Ожидание предсказания $E_X[\mu(X)(r_1)]$

Обучающая выборка «определяется» значением γ , поэтому $E_X[\cdot] = E_\gamma[\cdot]$:

$$E_\gamma[\mu(X)(r_1)] = E_\gamma[0.8 r_1 + 0.6 \gamma] = 0.8 r_1 + 0.6 \cdot E[\gamma] = 0.8 r_1$$

Смещение

$$\text{Bias} = E_x[(E_X[\mu(X)(x)] - E[y|x])^2] = E_{r_1}[(0.8 r_1 - 0.8 r_1)^2] = E_{r_1}[0] = 0$$

Разброс

$$\text{Variance} = E_x[E_X[(\mu(X)(x) - E_X[\mu(X)(x)])^2]]$$

Вычислим внутреннее ожидание:

$$\mu(X)(r_1) - E_\gamma[\mu(X)(r_1)] = (0.8 r_1 + 0.6 \gamma) - 0.8 r_1 = 0.6 \gamma$$

$$E_\gamma[(0.6 \gamma)^2] = 0.36 E[\gamma^2] = 0.36$$

Это не зависит от r_1 , поэтому:

$$\text{Variance} = E_{r_1}[0.36] = 0.36$$

Итого:

$$L(\mu) = \underbrace{0.36}_{\text{шум}} + \underbrace{0}_{\text{смещение}} + \underbrace{0.36}_{\text{разброс}} = 0.72$$

- **Шум** = 0.36 — это неустраняемая ошибка, обусловленная стохастичностью ε_2 в модели генерации данных.
- **Смещение** = 0 — в среднем по обучающим выборкам (по γ) предсказание метода совпадает с $E[r_2|r_1]$, так как $E[\gamma] = 0$. Метод является несмещённым.
- **Разброс** = 0.36 — при разных обучении (разных γ) предсказание отклоняется на 0.6γ от среднего. Дисперсия этого отклонения равна шуму, что логично: метод «подменяет» истинный шум ε_2 независимой копией γ с тем же распределением.